



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent relies on a lookup table that maps every possible historical sequence of percepts (P^*) directly to an action. In complex environments like Chess, storing this theoretical function is impossible because real computing hardware cannot hold a table with such a massive domain. As the agent's operating lifetime increases, its cumulative percept sequence grows longer, causing the number of

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules"

discussed in the lecture.

1. Environmental Condition-Action Rules (execute_action)

if tuple_new_pos in self.walls:

self.score -= 5

elif tuple_new_pos in self.toxic_traps:

self.score -= 15

self.agent_pos = new_pos

else:

self.agent_pos = new_pos

his block directly embodies the "if condition then action" structure. It evaluates the agent's proposed position against environmental rules (hitting a wall vs. stepping on a toxic trap) and immediately executes the corresponding rule (penalizing score, updating coordinates, or halting movement).

2. Percept Detection Rules (get_percept)

'smells_food': tuple(self.agent_pos) in self.food_positions,

'smells_toxin': tuple(self.agent_pos) in self.toxic_traps

Simple reflex agents rely on immediate sensory percepts rather than historical memory. These lines evaluate whether a condition in the environment is True or False at time t, producing the exact percept signals required to trigger a reflex action.

3. Actuator Transition Rules (execute_action)

if action == 'Up':

new_pos[1] = min(self.height - 1, new_pos[1] + 1)

elif action == 'Down':

new_pos[1] = max(0, new_pos[1] - 1)

3. Actuator Transition Rules (execute_action)

if action == 'Up':

new_pos[1] = min(self.height - 1, new_pos[1] + 1)

elif action == 'Down':

3. (Analyze) Your SimpleReflexAgent likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

Lack of Percept History (P*): Simple reflex agents process a direct mapping from current percepts to actions (f: P $\rightarrow$) rather than evaluating the complete historical percept sequence (P*). Because it does not store past states or previous actions in memory, the agent cannot detect that it has already visited a tile or executed a specific move.

4. (Evaluate) In Step 1.3, you added an internal state to your ModelBasedAgent . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

1. Transition Model (How the World Evolves)

Role: Represents the physics/rules of the world and predicts how the agent's actions modify the internal state.

Code Implementation: When `sense_and_act(percept)` receives the `last_action`, it computes expected position updates (e.g., if `last_action == 'Right'`, projected coordinate becomes $(x + 1, y)$). It models how moves translate into state changes regardless of immediate sensory inputs.

2. Sensor Model (How Percepts Reflect the World)

Role: Interprets current sensory data to confirm or correct the agent's internal state.

Code Implementation: Integrates incoming readings from `percept` (e.g., `agent_pos`, `hit_wall`, `smells_food`) directly into memory. For example, registering `self.visited_cells.add(current_pos)` or marking blocked coordinates when `hit_wall` evaluates to `True` updates internal knowledge based on real-time sensory feedback.

Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING